
Detection of Hateful Memes

Soheil Housni

Abstract

In the era of social media and online communication, hateful content is widely propagated, which can negatively impact users' sensitivity. However, human beings cannot manually filter all such content due to the enormous volume of data being generated. As a result, automated detection systems are necessary for detecting and filtering hateful content.

The Hateful Memes Challenge, introduced by Facebook AI (1), is a multimodal classification task. It consists of building a model capable of determining whether a meme is hateful or not. This challenge is specifically designed such that unimodal models are insufficient for this task, thereby requiring multimodal approaches that jointly process textual and visual information.

In this project, we propose a multimodal model that leverages a pretrained DistilBERT (7) model as a text encoder and a pretrained ResNet-18 (8) as a visual encoder. Our architecture incorporates a bidirectional cross-attention module (6) to integrate textual and visual representations by capturing interactions between them. We also experiment different fusion strategies for combining image and text embeddings, and evaluate the impact of incorporating CLIP embeddings.

Finally, we compare our approach with pretrained CLIP (3) and FLAVA (2) architectures, using only trainable classification heads. This allows us to assess the effectiveness of our trainable intermediate-fusion strategy against FLAVA's pretrained intermediate-fusion and CLIP's late-fusion of aligned embeddings, both of which are state-of-the-art multimodal models.

1 Introduction

In the era of social media and online communication, hateful content is widely propagated which can impact users' sensitivity. However, human beings cannot filter all hateful content due to the enormous flow of data that is circulating. As a result, automated detection systems are necessary for detecting and filtering hateful content.

The hateful meme challenge is a challenge issued by Meta about multimodal classification (1). It is a classification problem consisting in creating a model able to determine whether a meme is hateful or not. The particularity of this challenge is the difficulty of the dataset: Meta included in the dataset what they call "benign confounders" (1). Thus, for every hateful meme in the dataset, a new one is created by changing either only the image or only the caption of the meme in a way that its classification label flips to "not hateful", which makes the dataset ambiguous. In consequence, unimodal reasoning, processing either only text information or only image information, is insufficient for this task. Multimodal models processing text and image signals jointly are required to perform a good classification.

In this project, we propose a multimodal model that leverages a pretrained DistilBERT (7) model as a text encoder and a pretrained ResNet-18 (8) as a visual encoder. Our architecture incorporates a bidirectional cross-attention (6) module to integrate textual and visual representations by capturing interactions between them. This architecture is inspired by FLAVA (2) as it processes image and text sequences separately while still using cross-attention to capture their interactions. It also uses the

same logic as MMBT-Grid to extract the "visual tokens" from the image based on the cells of its feature maps.

In a second place, we experimented different fusion strategies of image and text embeddings: fusion them by doing the mean of both, or by concatenating them along with their product and absolute difference embeddings. We also evaluated the effect of incorporating CLIP embeddings (3) in our model. Indeed, CLIP embeddings can bring information about the alignment and the contrast of the text and the image. The hateful nature of a meme is often related to the contrast of its image and its caption. CLIP embeddings could then bring additional information about this aspect.

We finally compared this model to pretrained CLIP (3) and FLAVA (2) architectures using only trainable classification heads, and we could thus evaluate our trainable intermediate-fusion against the pretrained intermediate-fusion of FLAVA and the late-fusion of pretrained aligned CLIP embeddings, CLIP and FLAVA being both state-of-the-art multimodal models.

Our contributions are threefold: (1) we propose a hybrid multimodal architecture inspired by FLAVA (2) and MMBT-Grid, incorporating a trainable cross-attention module (6), (2) we investigate multiple fusion strategies for integrating image and text embeddings, and (3) we evaluate the impact of incorporating CLIP embeddings (3) into the model.

2 Related Work

At the time the challenge was released by Facebook AI (1), different multimodal models were trained. There were first early and intermediate fusion models such as ViLBERT (4), which uses two distinct streams for the text sequence and the visual sequence, each of those streams being composed of a series of transformer blocks that are connected through co-attentional transformer layers, and VisualBERT (5), which processes image and text tokens in a single sequence through a series of shared transformer blocks using a self-attention mechanism. Using this same process, there are also MMBT-Grid and MMBT-Region models, which extract visual tokens from the image respectively from the cells of feature maps retrieved from a CNN and from image regions obtained via a R-CNN.

Along with those models, the authors of the challenge (1) also tried late fusion models. The first one is a Late Fusion model that outputs the mean of the unimodal ResNet-152 and BERT model output scores. The second one is a Concat model that concatenates ResNet-152 and BERT representations, this concatenation then being fed to a MLP at the top.

More recent multimodal models have then been developed, such as FLAVA (2). This model employs distinct Image Encoder and Text Encoder modules to output hidden representations for respectively the image and the text, which are then concatenated and fed to a multimodal encoder composed of cross-attention blocks. The particularity of this model is that it has been trained on multiple multimodal objectives, such as Global Contrastive Loss (GCL), Masked Multimodal Modeling (MMM), and Image-Text Matching (ITM), along with multiple unimodal objectives such as Masked Language Modeling (MLM) and Masked Image Modeling (MIM). Thus, FLAVA performs well on a variety of multimodal and unimodal tasks.

A second recent multimodal model is CLIP (3), which produces text embeddings and image embeddings projected into the same embedding space such that embeddings of similar images and texts are brought closer and embeddings of dissimilar ones are kept apart, thanks to its training with a Contrastive Loss objective.

The purpose of our project is to explore a new hybrid multimodal architecture inspired by FLAVA (2) and MMBT-Grid. Our model leverages a pretrained DistilBERT (7) as a text encoder and a pretrained ResNet-18 (8) as an image encoder, while incorporating a trainable cross-attention (6) module for multimodal fusion. This design allows us to compare an intermediate fusion model trained specifically for hateful meme detection with recent state-of-the-art pretrained multimodal models such as FLAVA (2) and CLIP (3), where only the classification head is fine-tuned. Furthermore, unlike previous approaches, we investigate the impact of different fusion strategies of textual and visual representations. Finally, we evaluate whether the integration of pretrained CLIP embeddings (3) can further improve the performance of our trainable multimodal architecture.

3 Models architecture

To address the Hateful Meme challenge, we propose an architecture leveraging pretrained DistilBERT model (7) as text encoder and pretrained ResNet-18 (8) as visual encoder, and a bi-directional cross-attention module (6) to integrate both texts and images by capturing interactions between them. We then compared this model to pretrained CLIP (3) and FLAVA (2) architectures using only trainable classification heads.

3.1 DistilBERT_ResNet_model architecture

This model receives as input the meme caption and the meme image, and outputs the classification label that indicates whether the meme is hateful or not. This model can optionally take CLIP image and text embeddings (3) as input, as we aim to assess the impact of incorporating CLIP embeddings into our architecture.

3.1.1 Text-Processing

The meme caption is processed by a DistilBERT (7) transformer encoder, which produces a contextualized embedding for each token in the caption. Those embeddings are then normalized. We finally have a hidden representation of the text of dimension:

$(Sequence_length \times DistilBERT_hidden_dimension)$,

which is here:

$(Sequence_length \times 768)$.

3.1.2 Image-processing

The meme image is processed by the ResNet-18 (8) convolutional neural network. Before the average pooling layer, we extract the encoded image representation of dimension $(C \times H \times W)$, which is here $(512 \times 7 \times 7)$. As in MMBT-GRID, each spatial cell x_{ij} of the feature map is represented by a hidden embedding that contains its corresponding values across the C channels. Consequently, each image is represented as a sequence of hidden embeddings of dimension $((H \times W) \times C)$, which is here (49×512) . We then project these visual token embeddings into the same dimensional space as the text token embeddings (from 512 to 768) using a linear layer, before applying normalization. We therefore obtain an encoded image representation of dimension (49×768) .

3.1.3 Bidirectional Cross-Attention

In order to capture interactions between the meme image and its caption, we employ a cross-attention mechanism (6) to enrich their respective representations and improve their integration for the multimodal classification task. We thus obtain a better integration of them for our multimodal objective.

We first apply a cross-attention module to the textual representations, using the contextual text token embeddings as Queries and the visual token embeddings as Keys and Values. We then add the Cross-Attention embeddings obtained to the initial input text embeddings through a residual connection. The obtained text embeddings are then normalized and passed to a feed-forward network (FFN), which first projects the representations to half the dimensionality (from 768 to 384) before projecting them back to the original dimension (from 384 to 768). This FNN uses the GELU activation function and applies Dropout. Finally, after being passed to Dropout, the FFN output is added to its input through another residual connection, followed by a final normalization layer.

We implemented the same cross-attention module for the image, that in turn takes Image token embeddings as Queries, and Text contextual token embeddings as Keys and Values. We finally obtain enriched text and image token embeddings that capture interactions between both modalities.

3.1.4 Pooling

We then average the text token embeddings using the attention mask. This allows us to compute the average representation only over valid tokens while ignoring padding tokens. We thus obtain a single embedding representation of dimension (768) for the entire meme caption, which is then passed through a normalization layer.

In the same way, we average the image visual token embeddings to obtain a single embedding representation of dimension (768) for the meme image, which is then passed through a normalization layer.

3.1.5 Combination

In our implementation, we experimented two fusion strategies to integrate the text and image embeddings: the mean combination of these embeddings, and their concatenation along with their product and absolute difference embeddings.

In the case of mean fusion, we directly obtain a vector of dimension (768) that is then passed through a normalization layer.

In the case of concatenation fusion, along with the text and image embeddings, we concatenate the embeddings of their absolute difference:

$$[|text_embedding - image_embedding|]$$

and of their product:

$$[(text_embedding \times image_embedding)].$$

The former captures the distance between the two embeddings while the latter captures how they interact with each other. We thus have:

$$fusion_embedding = \begin{bmatrix} text_embedding \\ image_embedding \\ |text_embedding - image_embedding| \\ (text_embedding \times image_embedding) \end{bmatrix}$$

This embedding is of dimension $(768 \times 4 = 3072)$. We then project this concatenated embedding back to the original dimensionality (from 3072 to 768) before applying normalization.

3.1.6 Integration of CLIP embeddings

We sought to assess whether CLIP embeddings (3) could provide additional relevant information that would improve the performance of our model. Indeed, CLIP model is trained using a contrastive objective that enables it to bring closer similar image and text embeddings, and to keep apart dissimilar ones. Thus, CLIP embeddings can add information about the alignment and the contrast of the text and the image of a meme. Therefore, if the caption of a meme and its image are similar, their CLIP embeddings should be close in the embedding space, while if they are marked by a great difference they should be distant. The hateful nature of a meme is often related to the contrast of its image and its caption, and CLIP embeddings may thus bring useful information about this aspect.

Using a pretrained CLIP model, we retrieve the CLIP text and image embeddings corresponding respectively to the meme caption and image. We then create a new vector *clip_features* that concatenates the embeddings of the product and of the absolute difference of those CLIP embeddings. We thus have :

$$CLIP_features = \begin{bmatrix} (CLIP_image_embedding \times CLIP_text_embedding) \\ |CLIP_text_embedding - CLIP_image_embedding| \end{bmatrix}$$

We then project this *CLIP_features* embedding to the dimension of CLIP embeddings $(512 \times 2 = 1024 \text{ to } 512)$ before applying normalization. We finally concatenate this *clip_features* embedding with our *fusion_embedding* and we thus have: $CLIP_fusion_embedding = LayerNorm([fusion_embedding, clip_features])$ This embedding is of dimension $(768 + 512 = 1280)$.

3.1.7 FFN

The input to our FFN is directly the *fusion_embedding* if we don't use CLIP embeddings, or the *CLIP_fusion_embedding* if we do so. We finally feed this input into a FFN consisting of a first linear layer that projects the representation to half the dimensionality (from 768 to 384 if we don't use

CLIP embeddings, and from 1280 to 640 if we incorporate CLIP embeddings). This is followed by a second linear layer that maps the representation to the number of classes (here, 2). The FFN applies layer normalization after the first linear transformation, followed by a GELU activation function and dropout regularization. The model outputs the logits for each of the two classes.

3.2 Head_Classifier_CLIP_model

The *Head_Classifier_CLIP_model* takes as input the CLIP image and text embeddings retrieved from the pretrained CLIP Model (3).

3.2.1 Combination

Those embeddings are concatenated along with their absolute difference and product embeddings and we thus have :

$$CLIP_input = \begin{bmatrix} CLIP_text_embedding \\ CLIP_image_embedding \\ |CLIP_text_embedding - CLIP_image_embedding| \\ (CLIP_text_embedding \times CLIP_image_embedding) \end{bmatrix}$$

This *CLIP_input* embedding is of dimension $(4 \times CLIP_dimension)$ which is here $(4 \times 512 = 2048)$

3.2.2 FFN

The *CLIP_input* embedding is then fed into a FFN composed of a hidden layer projecting it to half the dimensionality (from 2048 to 1024). This is followed by a second linear layer that maps the representation to the number of classes (here, 2). The FFN applies layer normalization after the first linear transformation, followed by a GELU activation function and dropout regularization. The model outputs the logits for each of the two classes.

3.3 Head_Classifier_FLAVA_model

The *Head_Classifier_FLAVA_model* takes as input either the multimodal embeddings or the pooler embedding retrieved from the pretrained FLAVA model (2). The multimodal embeddings are the hidden representation of the text and image tokens outputted from the multimodal encoder of FLAVA. The pooler embedding is the hidden representation of the [CLS_M] multimodal token also outputted from the multimodal encoder. This model can optionally take CLIP image and text embeddings as input, as we aim to assess the impact of incorporating CLIP embeddings into our architecture.

3.3.1 Combination

In the case where the model takes as input the multimodal embeddings, we average them to obtain a single *FLAVA_input* embedding of dimension (768). Otherwise, the *FLAVA_input* embedding is directly the pooler embedding itself of dimension (768). If CLIP embeddings are received as input, we create the same *CLIP_features* embedding as the one used in the *DistilBERT_ResNet_model* architecture :

$$CLIP_features = \begin{bmatrix} (CLIP_image_embedding \times CLIP_text_embedding) \\ |CLIP_text_embedding - CLIP_image_embedding| \end{bmatrix}$$

We then project this *CLIP_features* embedding to the dimension of CLIP (from 1024 to 512) and we normalize it. We finally concatenate this *CLIP_features* embedding with the *FLAVA_input* embedding and we thus have:

$$FLAVA_CLIP_input = [FLAVA_input, CLIP_features]$$

This embedding is of dimension $(768 + 512 = 1280)$.

3.3.2 FFN

The input to the FFN is directly our *FLAVA_input* embedding if no CLIP embeddings are used, or the *FLAVA_CLIP_inputembedding* if we do so. The input embedding is then fed to a FFN

composed of a hidden layer projecting it to half the dimensionality (from 768 to 384, or from 1280 to 640 if CLIP embeddings are used), a normalization layer, and using GELU activation function and dropout. The model outputs the logits for each of the two classes.

4 Experimental Setup

4.1 Preparation of the Dataset

The dataset used is composed of 10 000 pairs of Meme image and captions (1). We split this dataset into 3 sets: a Train set composed of 7650 examples, a validation set composed of 500 examples, and a test set composed of 850 examples.

4.2 Hyperparameters

For the training of all the models, we use the same hyperparameters to be able to make useful comparisons.

For the optimizer, we use AdamW with a learning rate of 0.01, and a weight decay of 5e-4 for the L2 regularization.

For regularization, we also use an Early Stopping mechanism on the F1 metric in the direction of maximization with a patience of 2 epochs and a minimum improvement of 1e-3. Moreover, we have a dropout probability of 0.3 for the FFN, which is the same as the dropout probability used within the cross-attention module.

We use a linear scheduler for the learning rate with a warmup proportion of 10% of the total number of training steps.

As a loss function, we use the Cross Entropy Loss weighted with the class weights computed using the Scikit-Learn formula: $\frac{n_samples}{(n_classes \times count(c))}$

Our training lasts for 11 epochs, and the batch size used for training is 32.

5 Results

In order to compare the performance of the models, the two main metrics that we use are F1 and AUROC. Indeed, the training set, before being split to form the test set, is unbalanced, with approximately 64% of non-hateful labels and 36% of hateful labels. Thus, the F1 score, which integrates both Recall and Precision, is a good metric to assess the performance of the models in this unbalanced setting. The AUROC score is also an important metric for binary classification, as it measures the True Positive Rate and the False Positive Rate across all classification thresholds between 0 and 1.

Model	Cross Entropy Loss	Accuracy	F1	AUCROC
DistilBERT_ResNet_model architectures				
Concatenation model	0.58	0.72	0.72	0.78
Concatenation with CLIP embedding model	0.59	0.66	0.66	0.77
Mean fusion model	0.61	0.66	0.67	0.73
Mean fusion with CLIP embeddings model	0.58	0.67	0.68	0.77
Head_Classifier_CLIP_model architecture				
Head_Classifier_CLIP_model	0.58	0.70	0.70	0.76
Head_Classifier_FLAVA_model architecture				
Pooler_embedding model	0.83	0.64	0.50	0.51
Pooler_embedding with CLIP embeddings model	0.65	0.62	0.63	0.63
Multimodal_embeddings model	0.66	0.64	0.64	0.65
Multimodal_embeddings with CLIP embeddings model	0.65	0.66	0.65	0.66

Table 1: Comparison of multimodal models

The highest F1 score is obtained by the *DistilBERT_ResNet_model* using the concatenation-based fusion strategy that combines text and image embeddings with their absolute difference and element-wise product, without incorporating CLIP embeddings. It achieves a F1 score of 0.72 and an AUROC score of 0.78. This may be explained by the fact that more information is provided compared to simply averaging the text and image embeddings.

We also observe that, for this *DistilBERT_ResNet_model*, the integration of CLIP embeddings (3) has contrasting effects. For the Concatenation-based model, this integration has a negative impact, as the F1 score decreases from 0.72 to 0.66. The AUROC remains relatively stable, with a slight decrease from 0.78 to 0.77. In contrast, for the model using the Mean fusion strategy, the integration of CLIP embeddings has a positive effect, particularly improving the AUROC score from 0.73 to 0.77. This may be explained by the fact that CLIP embeddings capture additional information about the alignment between text and the image modalities that is not preserved by the mean fusion strategy, leading to improved performance. Conversely, such information may already be captured by the Concatenation-based fusion strategy, and adding CLIP embeddings (3) could introduce redundancy, which may degrade performance.

The performance of the *Head_Classifier_CLIP_model* leveraging a pretrained CLIP architecture (3) is close to that of the best *DistilBERT_ResNet_model*, although still slightly lower. Therefore, the intermediate fusion approach using a trainable Cross-Attention module is not significantly better than the late fusion of pretrained CLIP embeddings. This may suggest that the contrastive information provided by CLIP is significant for the classification of hateful memes. Moreover, since CLIP was pretrained on 400M image-text pairs (3), it achieves particularly strong performance when it comes to zero-shot transfer to downstream tasks.

For the *Head_Classifier_FLAVA_model* leveraging a pretrained FLAVA architecture (2), we can see that the integration of CLIP embeddings has a positive impact on the performance of the model, although the overall performance remains relatively low compared to the other models.

6 Discussion and Limitation

In conclusion, our *DistilBERT_ResNet_model* using the concatenation fusion strategy provides the best performance, although it remains close to that of the *Head_Classifier_CLIP_model*. Thus, our trainable intermediate fusion only marginally outperforms the late fusion approach based on pretrained CLIP embeddings. However, the *Head_Classifier_FLAVA_model* is the model that performs the worst. Overall, the impact of incorporating CLIP embeddings appears to be mixed, as their effectiveness depends on the fusion strategy and the degree of redundancy with existing representations.

Our training was limited by computational resources, and we were only able to experiment with a single set of hyperparameters. As a result, the reported performances are not necessarily optimal, and further improvements could likely be achieved through more extensive hyperparameter tuning. Moreover, the dataset contains only 10,000 examples, which limits the generalization capacity of the models and increases the risk of overfitting. Finally, due to computational limitations, we were unable to unfreeze and fine-tune some of the last layers of the ResNet-18 or DistilBERT encoders to our specific task, which could have further improved performance.

Therefore, future work for this project includes systematic hyperparameter tuning and a deeper investigation into whether fine-tuning the layers of the pretrained encoders can further improve performance. Additionally, data augmentation strategies could be explored to generate additional meme image-caption pairs, with the goal of improving the model’s robustness and generalization capabilities.

References

- [1] Kiela, D. and Firooz, H. and Mohan, A. (2020) The Hateful Memes Challenge: Detecting Hate Speech in Multimodal Memes. *arXiv preprint* arXiv:2005.04790. Available at <https://arxiv.org/abs/2005.04790>.
- [2] Singh, A. and Hu, R. and Goswami, G. and Couairon, G. and Galuba, W. and Rohrbach, M. and Kiela, D. (2021) FLAVA: A Foundational Language And Vision Alignment Model. *arXiv preprint* arXiv:2112.04482. Available at <https://arxiv.org/abs/2112.04482>.

- [3] Radford, A. and Kim, J. and Hallacy, C. and Ramesh, A. and Goh, G. and Agarwal, A. and Sastry, G. and Askeel, A. and Mishkin, P. and Clark, J. and Krueger, G. and Sutskever, I. (2021) Learning Transferable Visual Models From Natural Language Supervision. *arXiv preprint* arXiv:2103.00020. Available at <https://arxiv.org/abs/2103.00020>.
- [4] Lu, J. and Batra, D. and Parikh, D. and Lee, S. (2019) ViLBERT: Pretraining Task-Agnostic Visiolinguistic Representations for Vision-and-Language Tasks. *arXiv preprint* arXiv:1908.02265. Available at <https://arxiv.org/abs/1908.02265>.
- [5] Li, L. and Yatskar, M. and Yin, D. and Hsieh, C. and Chang, K. (2019) VisualBERT: A Simple and Performant Baseline for Vision and Language. *arXiv preprint* arXiv:1908.05535. Available at <https://arxiv.org/abs/1908.05535>.
- [6] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., and Polosukhin, I. (2017). Attention Is All You Need. *arXiv:1706.03762*. Available at <https://arxiv.org/abs/1706.03762>.
- [7] Sanh, V. and Debut, L. and Chaumond, J. and Wolf, T. (2019) DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter *arXiv preprint* arXiv:1910.01108. Available at <https://arxiv.org/abs/1910.01108>.
- [8] He, K. and Zhang, X. and Ren, S. and Sun, J. (2015) Deep Residual Learning for Image Recognition *arXiv preprint* arXiv:1512.03385. Available at <https://arxiv.org/abs/1512.03385>.